AP React – Partie 4
Ajout Modification Suppression

Introduction

Nous allons compléter notre application pour en faire une application CRUD (Create-Read-Update-Delete) complète. Nous avons la fonctionnalité d'affichage (Read), il nous manque l'ajout (CREATE), la modification (UPDATE) et la suppression (DELETE).

Nous allons utiliser la structure de fichiers suivante. Les fichiers en gras seront à créer, les autres ont déjà été créés, nous les modifierons seulement :

```
src/
|— components/
|        |— FraisForm.jsx      # Formulaire partagé pour l'ajout et la modification
|        └— FraisTable.jsx      # Tableau d'affichage avec boutons de modification/suppression
|— pages/
|        |— Dashboard.jsx      # Page principale avec la table
|        └— FraisEdit.jsx      # Page pour la modification
|— context/
|        └— FraisContext.js    # Contexte pour gérer les frais (optionnel)
└— styles/
         |— FraisForm.css      # Style pour le formulaire
         └— FraisTable.css     # Style pour la table
```

Nous utiliserons les méthodes et endpoint API suivants :

Fonctionnalité	Composant/Page	Méthode HTTP	Endpoint API	Body requête API
Lister les frais	**FraisTable.jsx**	**GET**	**/api/frais/liste/{idVisiteur}**	
Ajouter un frais	**FraisAdd.jsx + FraisForm.jsx**	**POST**	**/api/frais/ajout**	`{ "anneemois": "202412", "id_visiteur": 12, "nbjustificatifs": 4 }`
Modifier un frais	**FraisEdit.jsx + FraisForm.jsx**	**POST**	**/api/frais/modif**	`{ "id_frais": 49, "anneemois": "202412", "id_visiteur": 1, "nbjustificatifs": 4, "montantvalide": 125, "id_etat": 4 }`
Supprimer un frais	**FraisTable.jsx**	**DELETE**	**/api/frais/suppr**	`{ "id_frais": 49, }`

Ajout d'une note de frais

Etape 1 : Créer un formulaire partagé

Ce formulaire sera utilisé pour l'ajout et la modification des frais.

1. Créer un fichier FraisForm.jsx dans src/components/ :
   - Ce composant contiendra un formulaire avec les champs suivants :
     - anneeMois (ex: "202310").
     - nbJustificatifs (nombre entier).
     - montant (nombre décimal).
   - Le mieux est d'utiliser des états locaux (useState) pour gérer les valeurs des champs. Nous aurons besoin des états suivants, qu'il faut déclarer et initialiser au début de votre composant : idFrais, anneeMois, nbJustificatifs, montant, loading, error. idFrais peut être initialisé à null, loading à false, et les autres avec une chaine de caractères vide.
   - Créez un bouton dynamique, qui affiche Ajouter si loading vaut false et Enregistrement si loading est vrai (nous gérerons l'état loading plus tard) :

```
<button type="submit" disabled={loading}>
   {loading ? 'Enregistrement...' : 'Ajouter'}
</button>
```

2. Créer une page FraisAdd.jsx dans src/pages/. Cette page utilisera le composant FraisForm. Vous pouvez vous inspirer de la page Dashboard qui utilise le composant FraisTable.

Étape 2 : Gérer la soumission du formulaire

1. Si vous ne l'aviez pas fait, ajouter `onSubmit={handleSubmit}` dans votre balise <form>. Ainsi, à la soumission du formulaire, la fonction handleSubmit sera appelée.
2. Implémenter la fonction asynchrone handleSubmit dans FraisForm.jsx (vous avez déjà implémenté une fonction handleSubmit dans pages/Login.jsx) :
   - Empêcher le rechargement de la page avec

```
e.preventDefault();
```

   - Affecter la valeur true à loading et '' à error (avec leur méthode setLoading et setError)
   - Ecrire un bloc try/catch/finally.
     1. Dans le bloc try :
        - Récupérer le token depuis localStorage, nous en aurons besoin pour envoyer notre requête à l'API car elle est sécurisée. S'il n'est pas défini, lever une exception :

```
const token = localStorage.getItem('token');
if (!token) throw new Error('Token manquant');
```

        - Créer l'objet JSON à envoyer à l'API :

```
const fraisData = {
```

```
        anneemois: anneeMois,
        nbjustificatifs: parseInt(nbJustificatifs, 10),
        id_visiteur: getCurrentUser()["id_visiteur"]
};
```

- Envoyer les données à l'API avec axios.post :

```
const response = await
axios.post(`${API_URL}frais/ajout`, fraisData, {
          headers: { Authorization: `Bearer ${token}` },
        });
        console.log(response)
```

- Rediriger vers la liste des frais après succès :

```
navigate('/dashboard');
```

2. Dans le bloc catch : gérer les erreurs
   - Afficher l'erreur dans la console

```
console.error('Erreur:', err);
```

   - Mettre à jour l'état error avec un message d'erreur complet :

```
setError(err.response?.data?.message || err.message ||
'Erreur lors de l\'enregistrement');
```

3. Dans le bloc finally :
   - Mettre à jour l'état loading à false. L'enregistrement est terminé.

Étape 3 : Configurer le routage et tester

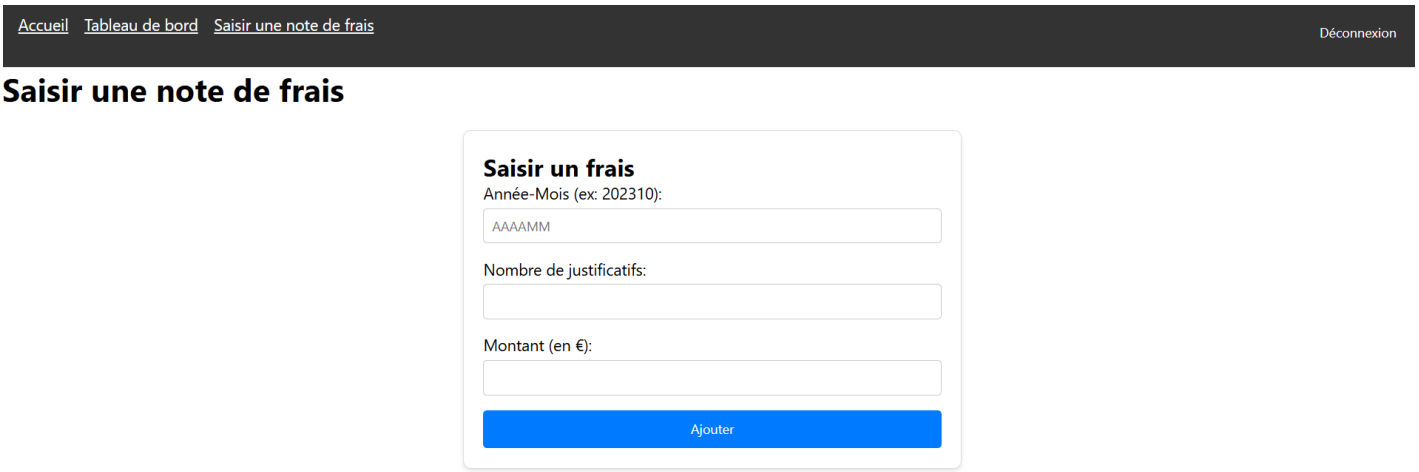
1. Dans App.js :
   - Importer le composant FraisAdd
   - Ajouter la route :

```
<Route path="/frais/ajouter" element={<FraisAdd />} />
```

2. Ajouter un lien dans la barre de navigation (composant Navbar) qui redirige vers /frais/ajouter si l'utilisateur est authentifié.
3. Tester le formulaire :
   - Vérifier que les données sont bien envoyées à l'API et ajoutées dans la base.
   - Vérifier que la redirection fonctionne après l'ajout.

Etape 4 : Gérer le style

Récupérer le fichier FraisForm.css fourni. Ajouter les valeurs de className aux balises de votre formulaire pour obtenir le rendu suivant :



Modification d'une note de frais

Etape 1 : Ajouter un bouton de modification dans le tableau de bord

1. Dans FraisTable.jsx, ajouter une colonne Actions dans le tableau des frais et ajouter dans cette colonne un bouton modifier :

```
<td>
   <button onClick={() => navigate(`/frais/modifier/${frais.id_frais}`)}
           className="edit-button"
   >
     Modifier
   </button>
</td>
```

L'event listener onClick permet de rediriger l'utilisateur vers la route /frais/modifier/{idFrais} lorsqu'il clique sur le bouton.

2. Dans FraisTable.css, ajouter le code suivant pour obtenir le rendu suivant :

```
.edit-button {
  background-color: #ffc107;
  margin-right: 5px;
}

.edit-button:hover {
  background-color: #e0a800;
}
```

ID	ID État	Année-Mois	ID Visiteur	Justificatifs	Date Modification	Montant saisi	Montant Validé	Actions
31	1	201708	2	5	2025-10-29T00:00:00.000000Z		250 €	Modifier
28	2	201710	2	3	2017-06-02T00:00:00.000000Z		349 €	Modifier
29	2	201702	2	2	0206-05-02T00:00:00.000000Z		349 €	Modifier

Etape 2 : Configurer le routage

Dans App.js :

- Importer le composant FraisEdit
- Ajouter la route :

```
<Route path="/frais/modifier/:id" element={<FraisEdit />} />
```

A ce stade, nous avons programmé le fait que lorsque l'utilisateur clique sur le bouton Modifier, il est redirigé vers la route /frais/modifier/{idFrais} qui affiche le composant FraisEdit.

Le composant FraisEdit est un composant « intermédiaire » qui va récupérer les données du frais à modifier pour les transmettre au composant FraisForm, le formulaire d'ajout/modification créé précédemment.

Il y a une **relation parent-enfant** entre les composants FraisEdit et FraisForm.

1. FraisEdit est le parent

- **FraisEdit** est un composant **parent** qui gère :
  - La récupération des données du frais à modifier depuis l'API.
  - L'état de chargement (loading).
  - L'affichage conditionnel (chargement, erreur, succès).

  Le passage des données (frais) au composant enfant : <FraisForm frais={frais} />

2. FraisForm est l'enfant

- **FraisForm** est un composant **enfant** qui reçoit les données (frais) depuis son parent (FraisEdit) via une **prop** : function FraisForm ({ frais = null }) { /* code du composant */ }
  - Il affiche un formulaire pré-rempli avec les données du frais (frais).
  - Il gère la soumission du formulaire (modification ou ajout).

Cette relation nous permet d'utiliser un seul formulaire pour ajouter et modifier un frais et de séparer les responsabilités : FraisEdit gère la **récupération des données**, FraisForm gère l'**affichage et la soumission du formulaire**. Le code sera ainsi plus maintenable (par exemple, si on veut modifier le formulaire, on ne modifiera que FraisForm).

Etape 3 : Implémenter le composant FraisEdit

1. Créer un fichier FraisEdit.jsx dans src/components/
2. Dans FraisEdit.jsx, importer les élements dont nous aurons besoin :

- useState pour gérer l'état local du composant, useEffect pour exécuter les effets de bord (comme un appel API) après le rendu du composant :
- axios pour faire notre requête get à l'API pour récupérer les informations du frais
- le hook useParams de react-router-dom pour récupérer le paramètre id de l'URL (id dans /frais/modifier/:id)

```
import { useParams } from 'react-router-dom';
```

- le composant FraisForm

3. Déclarer le composant FraisEdit et l'implémenter comme suit :
   - Récupérer l'id de l'URL :

```
const { id } = useParams();
```

   - Déclarer et initialiser les états locaux : frais (initialisé à null) qui stockera les données du frais à modifier, loading (initialisé à true) qui indique si les données sont en cours de chargement.
   - Récupérer les données du frais au montage du composant avec le hook useEffect :

```
useEffect(() => {
    const fetchFrais = async () => {
      try {
        const token = localStorage.getItem('token');
        const response = await axios.get(`${API_URL}frais/${id}`, {
          headers: { Authorization: `Bearer ${token}` },
        });
        setFrais(response.data);
      } catch (error) {
        console.error('Erreur:', error);
      } finally {
        setLoading(false);
      }
    };
    fetchFrais();
  }, [id]);
```

useEffect : Exécute le code à l'intérieur **une fois** (au montage du composant) et à chaque fois que id change (grâce au tableau de dépendances [id]).
fetchFrais : Fonction asynchrone qui récupère les données du frais depuis l'API.
axios.get :

- Envoie une requête **GET** à l'API pour récupérer le frais avec l'ID spécifié.
- Le token d'authentification est ajouté dans les en-têtes (Authorization: Bearer <token>).

setFrais(response.data) : Met à jour l'état frais avec les données reçues de l'API.
finally : setLoading(false) est appelé dans tous les cas (succès ou erreur) pour indiquer que le chargement est terminé.

4. Gérer l'affichage conditionnel du message :
   - si le chargement est en cours, on affiche « Chargement… »

```
if (loading) return <div>Chargement...</div>;
```

   - si frais est null, c'est-à-dire que l'expression !frais vaut true (par exemple, si l'API renvoie une erreur ou si le frais n'existe pas), on affiche « Frais non trouvé »
5. Affichage du composant : renvoyer le composant FraisForm, en lui passant frais comme prop.
6. Ne pas oublier d'exporter le composant.

Etape 4 : Modifier le formulaire pour gérer le cas de la modification

1. Dans le cas d'une modification de frais, le clic sur le bouton déclenche la génération du composant FraisEdit qui déclenche la génération du composant FraisForm, en lui passant frais comme prop. Il faut alors pré-remplir le formulaire avec les informations du frais, en utilisant le hook useEffect, déclenché à chaque modification de la prop frais. Copier le code suivant dans le composant FraisForm (après la déclaration des état et avant la fonction handleSubmit) et le compléter pour pré-remplir aussi les champs anneeMois et nbJustificatifs :

```
// Pré-remplir le formulaire si on modifie un frais existant
  useEffect(() => {
    if (frais) {
      setIdFrais(frais.id_frais);
      setMontant(frais.montantvalide || '');
      // TODO : compléter en affectant la valeur à anneeMois et
nbJustificatifs
    }
  }, [frais]);
```

2. Il faut maintenant modifier la fonction handleSubmit() pour gérer le cas de la modification : la requête API n'est pas la même que dans le cas d'un ajout.
   - Supprimer le code permettant de créer fraisData et de faire la requête axios.
   - Il faut modifier le contenu du body envoyé à la requête : notre objet fraisData. Dans le cas de l'ajout, on a besoin de id_visiteur, anneemois, nbjustificatifs. Dans le cas de la modification, on a besoin de id_frais, anneemois, nbjustificatifs, montantvalide. Commencons par créer un objet fraisData contenant les deux attributs communs aux deux cas : anneemois et nbjustificatifs.

```
      const fraisData = {
        anneemois: anneeMois,
        nbjustificatifs: parseInt(nbJustificatifs, 10),
      };
```

   - Ensuite distinguons les deux cas avec un if/else. Copier et compléter le code suivant :

```
if (frais) {// Mise à jour d'un frais existant (UPDATE)
   fraisData["id_frais"] = idFrais; // ajoute id_frais au JSON fraisData
```

```
    fraisData["montantvalide"] = parseFloat(montant);
    // TODO : compléter la requête
    const response = await axios.post(
       // TODO : passer l'url de modification (voir le tableau au début du doc),
       // TODO : passer l'objet JSON du body,
       // TODO : passer le token dans les headers
    );
    console.log(response)
} else { // Ajout d'un nouveau frais (CREATE)
    // TODO : Ajouter id_visiteur à fraisData
    const response = await axios.post(`${API_URL}frais/ajout`, fraisData, {

      headers: { Authorization: `Bearer ${token}` },
    });
    console.log(response)
}
```

1. Modifier le titre du formulaire : si frais n'est pas null, on veut afficher « Modifier le frais », sinon « Saisir un frais »

```
<h2>{frais ? 'Modifier le frais' : 'Saisir un frais'}</h2>
```

2. Modifier l'affichage du bouton : si frais n'est pas null, on veut un bouton « Mettre à jour », sinon un bouton « Ajouter »

```
<button type="submit" disabled={loading}>
   {loading ? 'Enregistrement...' : (frais ? TODO : compléter les 2 cas ici)}
</button>
```

4. Tester le formulaire :
   - Vérifier que les données sont bien envoyées à l'API et modifiées dans la base.
   - Vérifier que la redirection fonctionne après la modification.

Suppression d'une note de frais

Etape 1 : Ajouter un bouton de suppression dans le tableau de bord

Comme vous l'avez fait pour la modification, ajoutez un bouton Supprimer dans le tableau de bord. Celui-ci déclenchera l'exécution de la fonction `handleDelete`. Vous pourrez utiliser le CSS suivant pour obtenir le rendu ci-dessous :

```
.delete-button {
  background-color: #dc3545;
}
```

```
.delete-button:hover {
  background-color: #c82333;
}
```

ID	ID État	Année-Mois	ID Visiteur	Justificatifs	Date Modification	Montant saisi	Montant Validé	Actions
31	1	201708	2	5	2025-10-29T00:00:00.000000Z		250 €	Modifier Supprimer
28	2	201710	2	3	2017-06-02T00:00:00.000000Z		349 €	Modifier Supprimer
29	2	201702	2	2	0206-05-02T00:00:00.000000Z		349 €	Modifier Supprimer
30	2	201709	2	1	2017-11-09T00:00:00.000000Z		125 €	Modifier Supprimer

Etape 2 : Gérer la suppression

La suppression sera gérée directement dans FraisTable. Il n’y a pas besoin de composant supplémentaire ici.

Il suffit d’implémenter la fonction handleDelete dans le composant FraisTable. Copier et compléter la fonction fournie :

```
const handleDelete = async (id) => {
    if (!window.confirm('Êtes-vous sûr de vouloir supprimer ce frais ?')) return;

    try {
      await axios.delete(
            // TODO : passer l’url de suppression,
            {
                  data : {id_frais : /*TODO*/ },
                  headers : {Authorization : /*TODO*/ }
            }
      });
      // Met à jour FraisList en ignorant le frais qui a été supprimé : on ne
garde que les frais dont l’id est différent de l’id du frais sélectionné
      setFraisList(fraisList.filter((frais) => frais.id_frais !== id));
    } catch (error) {
      console.error('Erreur lors de la suppression:', error);
    }
  };
```

Conclusion

Vous avez maintenant une application CRUD complète, permettant de gérer des notes de frais. Dans la suite, nous allons améliorer les fonctionnalités pour que l'utilisateur ne saisisse pas le montant à la main mais saisisse des frais au forfait et/ou des frais hors forfait et que le montant soit calculé automatiquement.